

MECHATRONICS AND IOT

ASSIGNMENT REPORT – 3

TITLE:

Design and simulate a Smart Flood Level Monitoring System using Ultrasonic Sensor. The system continuously monitors water level and provides alerts during critical flood conditions for disaster management applications.

TEAM MEMBERS:

HARISANKARAN S
(24MER013)

MUTHUKUMAR M
(24MER034)

MONESHWARAN C
(24MER032)

PROJECT OVERVIEW

Floods are among the most common natural disasters and can cause significant damage to people, infrastructure, agriculture, and property. Early detection of rising water levels is important for disaster management and emergency response.

The proposed **Smart Flood Level Monitoring System** continuously measures water level using an **HC-SR04 ultrasonic sensor** connected to an **ESP32 microcontroller**. The ESP32 calculates the water level and classifies the condition as **SAFE, WARNING, or FLOOD**.

The measured water level and system status are transmitted through Wi-Fi to the **Adafruit IO Cloud**. A cloud dashboard provides remote monitoring and graphical visualization of the water-level data.

The system also provides local warning through a **green LED, red LED, OLED display, and buzzer**.

PROBLEM STATEMENT

Major problems include:

- Delayed detection of rising water levels.
- Lack of continuous water-level monitoring.
- Difficulty in remotely observing flood conditions.
- No automatic local warning system.
- Lack of historical water-level data.
- Delayed response during critical flood conditions.
- Limited accessibility to real-time measurements.

PROPOSED SOLUTION

- Measures the distance between the sensor and water surface.
- Calculates the water level.
- Calculates the water-level percentage.
- Classifies the condition as SAFE, WARNING, or FLOOD.
- Displays readings on the OLED.
- Controls the green LED during safe conditions.
- Controls the red LED during critical conditions.
- Activates the buzzer during flood conditions.
- Connects to Wi-Fi.
- Uploads sensor data to Adafruit IO.
- Provides a cloud dashboard for remote monitoring.
- Stores historical water-level readings in the cloud.

WORKING PRINCIPLE

1. The ESP32 is powered on.
2. The OLED, LEDs, buzzer, and ultrasonic sensor are initialized.
3. The ESP32 connects to the Wi-Fi network.
4. The ESP32 connects to Adafruit IO Cloud.
5. The HC-SR04 sends an ultrasonic pulse.
6. The pulse reflects from the water surface.
7. The Echo time is measured by the ESP32.
8. The distance to the water surface is calculated.
9. The water level is calculated from the container height.
10. The water-level percentage is calculated.
11. The percentage is compared with predefined thresholds.
12. The system determines SAFE, WARNING, or FLOOD status.
13. The OLED displays the current readings.
14. The green LED indicates safe conditions.
15. The red LED indicates critical flood conditions.
16. The buzzer activates during a flood condition.
17. Water level and distance are uploaded to Adafruit IO.
18. The cloud dashboard displays the data.
19. The process repeats continuously.

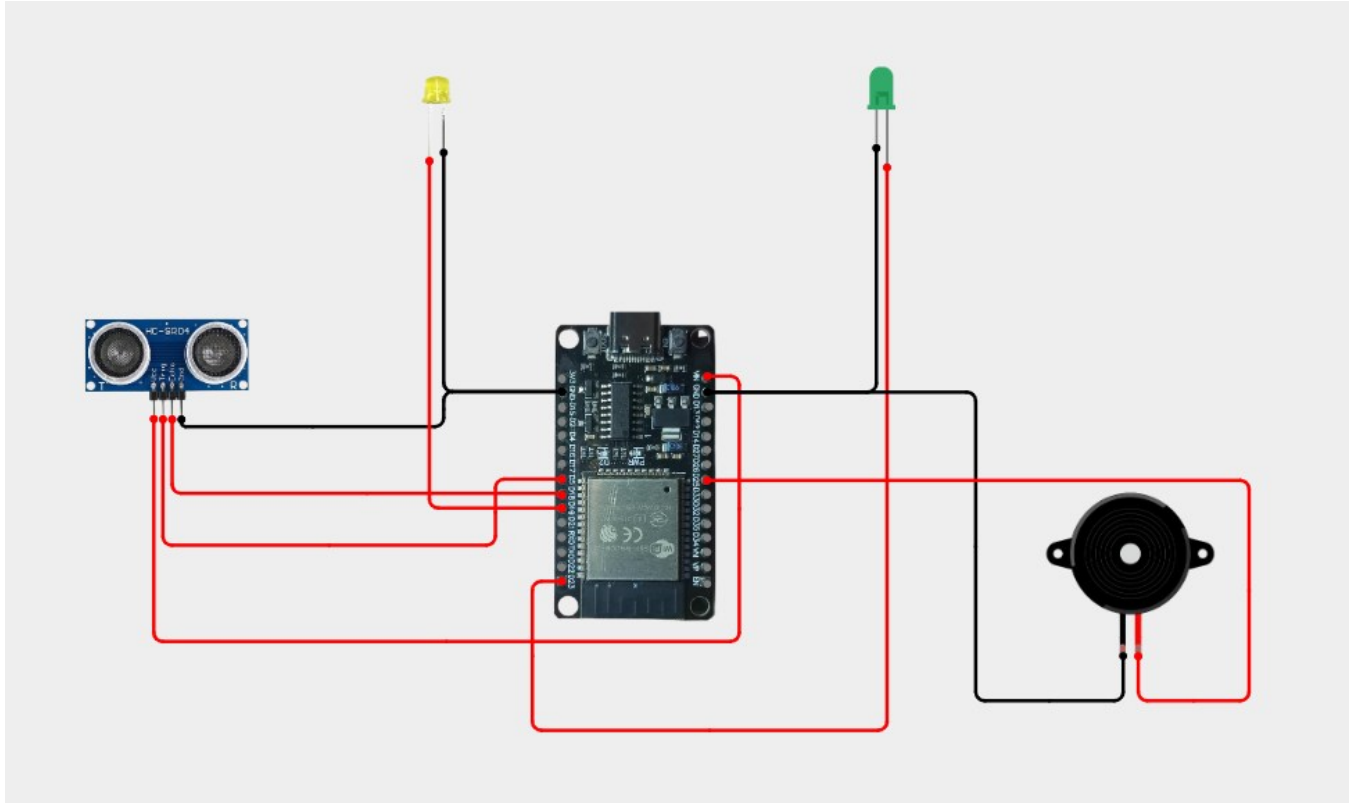
SYSTEM ARCHITECTURE

This follows the same initialization → measurement → threshold comparison → display → cloud publication repeat structure used in the reference report.

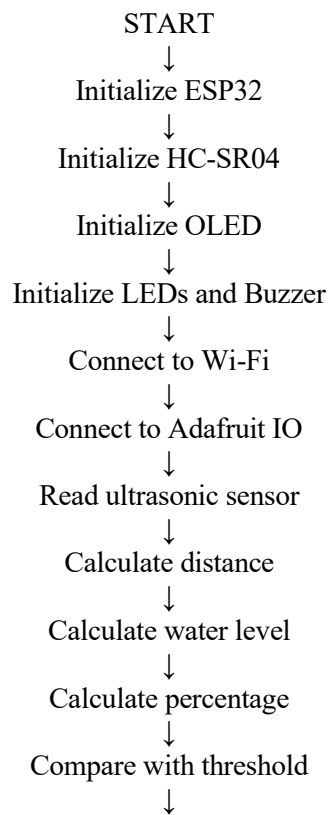
CIRCUIT TABLE

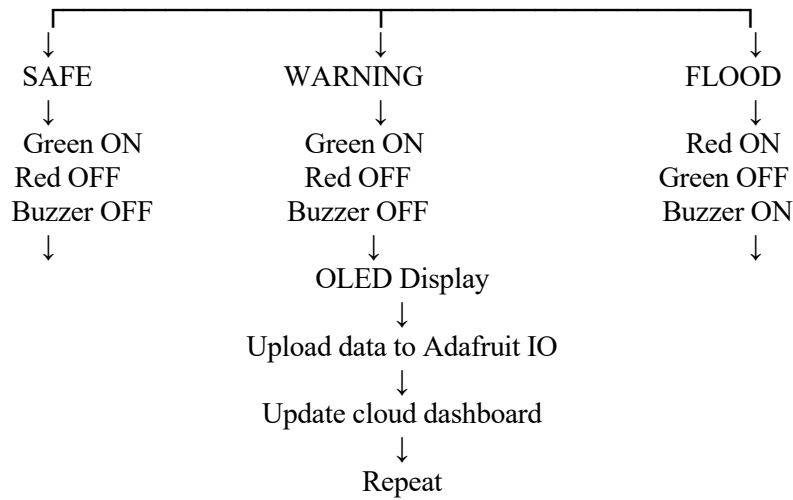
Component	Pin	ESP32 GPIO	Purpose
HC-SR04	TRIG	GPIO 5	Trigger pulse
HC-SR04	ECHO	GPIO 18	Distance measurement
OLED	SDA	GPIO 21	I2C data
OLED	SCL	GPIO 22	I2C clock
Green LED	Anode through resistor	GPIO 23	Safe indication
Red LED	Anode through resistor	GPIO 19	Flood indication
Buzzer	Positive	GPIO 25	Alarm
OLED	VCC	3.3V	Power
OLED	GND	GND	Ground
HC-SR04	VCC	5V/VIN	Power
HC-SR04	GND	GND	Ground

CIRCUIT DESIGN



ALGORITHM





CODING

```
// =====
// SMART FLOOD LEVEL MONITOR
// ESP32 + HC-SR04 + OLED + LEDs + Buzzer + Adafruit IO
// =====

#include <WiFi.h>

#include "AdafruitIO_WiFi.h"

#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

// =====
// ADAFRUIT IO SETTINGS
// =====

// PUT YOUR ADAFRUIT IO USERNAME HERE
#define IO_USERNAME "Hair_13"

// PUT YOUR ADAFRUIT IO KEY HERE
#define IO_KEY "aio_EMRC40vci7DQ5Amz5kvOKh6WKHI0"

// =====
// WIFI SETTINGS
// =====

// PUT YOUR WIFI NAME HERE
#define WIFI_SSID "OnePlus Nord CE5 7C5C"

// PUT YOUR WIFI PASSWORD HERE
#define WIFI_PASS "qkts5737"
```

```

// =====
// CREATE ADAFRUIT IO CONNECTION
// =====

AdafruitIO_WiFi io(
  IO_USERNAME,
  IO_KEY,
  WIFI_SSID,
  WIFI_PASS
);

// =====
// ADAFRUIT IO FEEDS
// =====

AdafruitIO_Feed *waterLevelFeed =
  io.feed("water-level");

AdafruitIO_Feed *distanceFeed =
  io.feed("distance");

AdafruitIO_Feed *statusFeed =
  io.feed("status");

AdafruitIO_Feed *floodAlertFeed =
  io.feed("flood-alert");

// =====
// ESP32 PIN CONNECTIONS
// =====

#define TRIG_PIN 5
#define ECHO_PIN 18

#define GREEN_LED 23
#define RED_LED 19

#define BUZZER_PIN 25

#define OLED_SDA 21
#define OLED_SCL 22

// =====
// OLED SETTINGS
// =====

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

#define OLED_ADDRESS 0x3C

```

```

Adafruit_SSD1306 display(
  SCREEN_WIDTH,
  SCREEN_HEIGHT,
  &Wire,
  -1
);

// =====
// TANK SETTINGS
// =====

// CHANGE THIS TO YOUR ACTUAL TANK/CONTAINER HEIGHT

const float TANK_HEIGHT = 30.0;

// WARNING LEVEL

const float WARNING_LEVEL = 50.0;

// FLOOD LEVEL

const float FLOOD_LEVEL = 80.0;

// =====
// FUNCTION: READ ULTRASONIC DISTANCE
// =====

float readDistance()
{
  digitalWrite(TRIG_PIN, LOW);

  delayMicroseconds(2);

  digitalWrite(TRIG_PIN, HIGH);

  delayMicroseconds(10);

  digitalWrite(TRIG_PIN, LOW);

  unsigned long duration =
    pulseIn(ECHO_PIN, HIGH, 30000);

  // Sensor timeout

  if (duration == 0)
  {
    return -1;
  }
}

```

```

// Convert time to distance

float distance =
    duration * 0.0343 / 2.0;

return distance;
}

// =====
// FUNCTION: SHOW SENSOR ERROR
// =====

void showSensorError()
{
    digitalWrite(GREEN_LED, LOW);

    digitalWrite(RED_LED, HIGH);

    digitalWrite(BUZZER_PIN, LOW);

    display.clearDisplay();

    display.setTextColor(SSD1306_WHITE);

    display.setTextSize(1);

    display.setCursor(15, 10);

    display.println("ULTRASONIC");

    display.setCursor(30, 25);

    display.println("SENSOR");

    display.setCursor(40, 40);

    display.println("ERROR!");

    display.display();
}

// =====
// FUNCTION: UPDATE OLED
// =====

void updateOLED(
    float distance,
    float percentage,
    const char *status
)

```



```

{
  display.clearDisplay();

  display.setTextColor(SSD1306_WHITE);

  // Title

  display.setTextSize(1);

  display.setCursor(0, 0);

  display.println("SMART FLOOD MONITOR");

  // Distance

  display.setCursor(0, 15);

  display.print("Distance: ");

  display.print(distance, 1);

  display.println(" cm");

  // Water percentage

  display.setCursor(0, 28);

  display.print("Water: ");

  display.print(percentage, 0);

  display.println("%");

  // Status

  display.setCursor(0, 42);

  display.print("Status: ");

  display.println(status);

  display.display();
}

```

```

// =====
// FUNCTION: SEND DATA TO ADAFRUIT IO
// =====

```

```

void sendData()

```

```

{
  Serial.println();
  Serial.println("-----");

  // Read sensor

  float distance = readDistance();

  // Sensor error

  if (distance < 0)
  {
    Serial.println("Ultrasonic sensor error!");

    showSensorError();

    return;
  }

  // =====
  // CALCULATE WATER LEVEL
  // =====

  float waterLevel =
    TANK_HEIGHT - distance;

  // Keep water level >= 0

  if (waterLevel < 0)
  {
    waterLevel = 0;
  }

  // Keep water level <= tank height

  if (waterLevel > TANK_HEIGHT)
  {
    waterLevel = TANK_HEIGHT;
  }

  // =====
  // CALCULATE PERCENTAGE
  // =====

  float percentage =
    (waterLevel / TANK_HEIGHT) * 100.0;

  // =====

```

```

// SAFE
// =====

if (percentage < WARNING_LEVEL)
{
    digitalWrite(GREEN_LED, HIGH);

    digitalWrite(RED_LED, LOW);

    digitalWrite(BUZZER_PIN, LOW);

    Serial.println("STATUS: SAFE");

    // Send status

    statusFeed->save("SAFE");

    // No flood alert

    floodAlertFeed->save(0);

    updateOLED(
        distance,
        percentage,
        "SAFE"
    );
}

// =====
// WARNING
// =====

else if (percentage < FLOOD_LEVEL)
{
    digitalWrite(GREEN_LED, HIGH);

    digitalWrite(RED_LED, LOW);

    digitalWrite(BUZZER_PIN, LOW);

    Serial.println("STATUS: WARNING");

    // Send status

    statusFeed->save("WARNING");

    // No flood alert

```

```

floodAlertFeed->save(0);

updateOLED(
  distance,
  percentage,
  "WARNING"
);
}

// =====
// FLOOD
// =====

else
{
  digitalWrite(GREEN_LED, LOW);

  digitalWrite(RED_LED, HIGH);

  digitalWrite(BUZZER_PIN, HIGH);

  Serial.println("STATUS: FLOOD!");

  // Send status

  statusFeed->save("FLOOD!");

  // Flood alert ON

  floodAlertFeed->save(1);

  updateOLED(
    distance,
    percentage,
    "FLOOD!"
  );
}

// =====
// SEND SENSOR DATA TO CLOUD
// =====

waterLevelFeed->save(percentage);

distanceFeed->save(distance);

```

```

// =====
// SERIAL MONITOR
// =====

Serial.print("Distance: ");

Serial.print(distance);

Serial.println(" cm");


Serial.print("Water Level: ");

Serial.print(waterLevel);

Serial.println(" cm");


Serial.print("Water Percentage: ");

Serial.print(percentage);

Serial.println("%");


Serial.println("-----");
}


// =====
// SETUP
// =====

void setup()
{
  Serial.begin(115200);

  delay(1000);

  Serial.println();

  Serial.println("=====");

  Serial.println("SMART FLOOD MONITOR");

  Serial.println("ESP32 + ADAFRUIT IO");

  Serial.println("=====");


// =====
// SENSOR PINS
// =====

```

```

pinMode(TRIG_PIN, OUTPUT);

pinMode(ECHO_PIN, INPUT);

// =====
// LED PINS
// =====

pinMode(GREEN_LED, OUTPUT);

pinMode(RED_LED, OUTPUT);

// =====
// BUZZER
// =====

pinMode(BUZZER_PIN, OUTPUT);

// Turn OFF outputs initially

digitalWrite(GREEN_LED, LOW);

digitalWrite(RED_LED, LOW);

digitalWrite(BUZZER_PIN, LOW);

// =====
// OLED
// =====

Wire.begin(
  OLED_SDA,
  OLED_SCL
);

if (!display.begin(
  SSD1306_SWITCHCAPVCC,
  OLED_ADDRESS))
{
  Serial.println("OLED NOT FOUND!");

  while (true)
  {
    delay(100);
  }
}

// Startup screen

```

```

display.clearDisplay();

display.setTextColor(SSD1306_WHITE);

display.setTextSize(2);

display.setCursor(10, 5);

display.println("SMART");


display.setCursor(10, 28);

display.println("FLOOD");


display.setTextSize(1);

display.setCursor(30, 52);

display.println("MONITOR");


display.display();


delay(2000);


// =====
// CONNECT TO ADAFRUIT IO
// =====


Serial.println();

Serial.println("Connecting to Adafruit IO...");


io.connect();


// Wait for connection
while (io.status() < AIO_CONNECTED)
{
  Serial.print(".");

  delay(500);
}


Serial.println();

Serial.println("Connected to Adafruit IO!");

```

```

// =====
// FIRST SENSOR READING
// =====

sendData();
}

// =====
// LOOP
// =====

void loop()
{
  // Keep Adafruit IO connection alive

  io.run();

  // Send data every 2 seconds

  static unsigned long lastSend = 0;

  if (millis() - lastSend >= 2000)
  {
    lastSend = millis();

    sendData();
  }
}

```


SYSTEM WORKING

The HC-SR04 continuously measures the distance between the sensor and the water surface. The ESP32 uses this measurement to calculate the water level.

For example, if:

Tank height = 30 cm

Measured distance = 9 cm

Then:

Water level = 30 - 9

Water level = 21 cm

Percentage:

Water level % = $(21 / 30) \times 100$

Water level % = 70%

Therefore:

70% → WARNING

The ESP32 will:

Green LED → ON

Red LED → OFF

Buzzer → OFF

OLED → WARNING

Cloud → WARNING

If the level reaches 80%:

Red LED → ON

Green LED → OFF

Buzzer → ON

OLED → FLOOD!

Cloud → FLOOD!

Alert → 1

BILL OF MATERIALS

S.No.	Component	Quantity
1	ESP32 Dev Board	1
2	HC-SR04 Ultrasonic Sensor	1
3	OLED Display 0.96" I2C	1
4	Green LED	1
5	Red LED	1
6	Buzzer	1
7	Breadboard	1
8	Jumper Wires	As required
9	Resistors for LEDs	2
10	Resistors for HC-SR04 Echo voltage divider	2
11	USB Cable	1
12	Wi-Fi connection	1

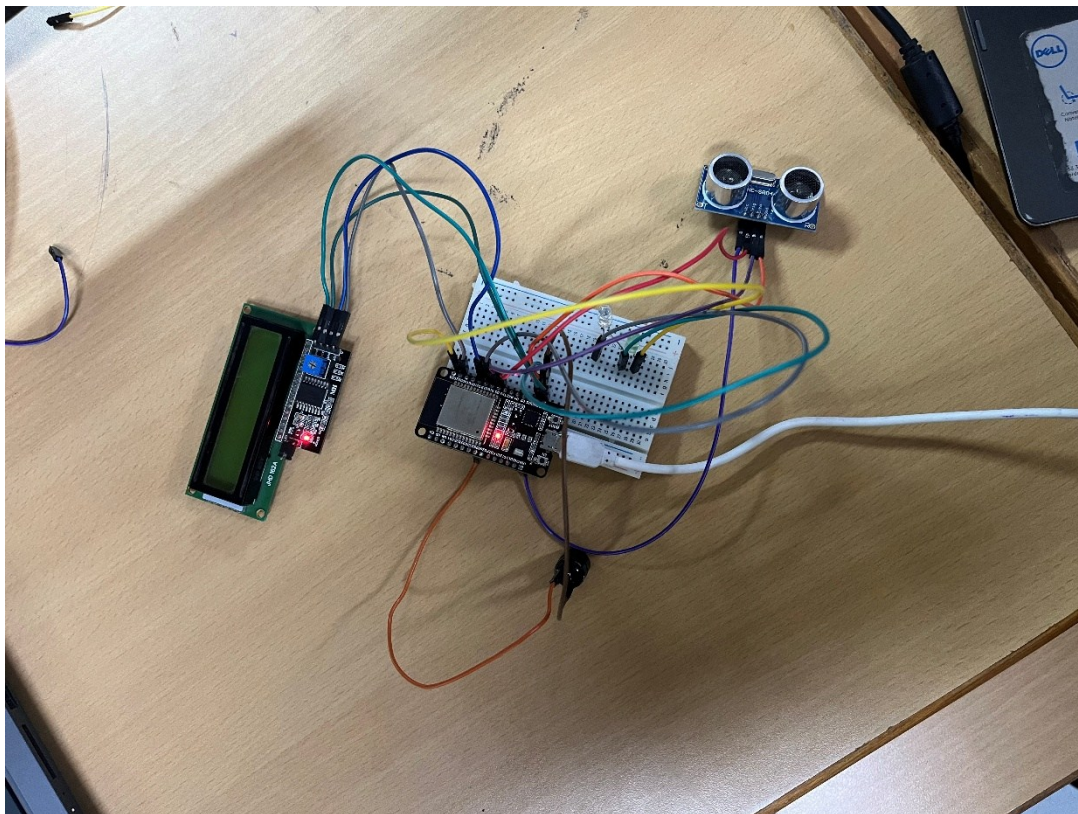
PROTOTYPE IMPLEMENTATION

The prototype is built using an ESP32 on a breadboard. The HC-SR04 ultrasonic sensor is positioned above a water container to measure the distance to the water surface. The OLED provides local display of distance, water level, and status.

Two LEDs provide visual status indication, while the buzzer provides an audible warning during critical flood conditions.

The ESP32 connects to Wi-Fi and transmits the measured data to Adafruit IO Cloud. The cloud dashboard allows the user to remotely monitor water-level conditions and observe historical trends.

The reference project similarly uses a breadboard-based ESP32 prototype, local display, warning indicator, wireless communication, and cloud visualization.



RESULTS & PERFORMANCE ANALYSIS

Water level	<50%	50–79%	≥80%
Green LED	ON	ON	OFF
Red LED	OFF	OFF	ON
Buzzer	OFF	OFF	ON
OLED	SAFE	WARNING	FLOOD
Cloud status	SAFE	WARNING	FLOOD
Flood alert	0	0	1

PERFORMANCE

- Real-time water-level monitoring.
- Ultrasonic-based non-contact measurement.
- Local OLED display.
- Green/red visual indication.
- Audible flood alarm.
- Wi-Fi connectivity.
- Adafruit IO cloud monitoring.
- Historical water-level visualization.
- Automatic flood-condition detection.
- Remote monitoring capability.
- Low-cost prototype implementation.

